

0.1 Introduction

Chapter 1 introduced three questions that can be asked of a process system modelled as a directed acyclic graph, and named the method that answers each. This chapter reviews those methods and the literature around them. Sections 0.2 to 0.4 cover network reliability, network flow and scheduling in turn, with attention to the exact methods and to where their cost comes from. Section 0.5 reviews the representations available for uncertain inputs, defines the two that this thesis uses, and records how each of the three literatures has treated imprecision. Section 0.6 examines the tools and methods that run more than one of the three analyses on a single network, and Section 0.7 the forms in which engineering analysis software reaches its users. Section 0.8 notes the state of graph tooling in the languages the work could have been written in. The chapter closes with the gap that the rest of the thesis addresses.

0.2 Network Reliability Computation

The reliability of a network in this work is the probability that the network performs its function when its components can fail. For the kind of systems the framework addresses, the function is delivery from a source to a node, so the quantity of interest is the probability that at least one path from a source to the node has all of its components operating. Computing this quantity exactly is $\#P$ -hard in general [1, 2], so every exact method is exponential in some property of the network, and the methods differ in which property that is.

The earliest formulations are structural. A reliability block diagram arranges components in series, parallel and k -out-of- n groups, each of which has a closed-form system reliability [3]. Fault trees and event trees build the failure mechanism explicitly as a tree of logic gates [4]. Both are efficient as far as the failure logic reduces to a tree or a series-parallel structure. However, both stop being efficient at the same point, which is where routes in the network diverge and reconverge with shared components upstream. This structure follows from the design of the system. A robust network provides more than one route to the same destination, and two routes to one destination begin at a common component. Hence the redundancy that makes a system worth analysing is also the structure these formalisms cannot hold [5, 6].

Graph-theoretic methods compute on the network directly. Since reaching a node is the union of its path events, the minimal path sets and the minimal cut sets give the reliability exactly once the union is made disjoint, and recursive decomposition improves on raw enumeration by splitting the network into subproblems [7, 8]. However, the collections of paths and cuts grow combinatorially with the same redundancy that motivates the analysis. Series-parallel reduction collapses independent stretches of the

network pair by pair [9], and where the network is not series-parallel, factoring conditions on a single component and recurses on the working and failed cases until the remainder becomes reducible [10–12]. Polygon-to-chain reductions widen what can be collapsed in one step beyond a plain series or parallel pair [13]. Tree decomposition generalises the idea to graphs that no sequence of such reductions can collapse. The network is arranged into small overlapping bags connected as a tree [14], and the cost of exact computation is then bounded by the width of that decomposition, whatever the size of the graph. This is what makes exact reliability feasible on networks as large as metropolitan transit systems [15]. Monte Carlo simulation does not depend on structure and handles any network, but it is not exact, and the sample count it needs grows large when failures are rare or the required precision is high [16].

Binary decision diagrams (BDDs) compile the Boolean structure function of the network into a canonical decision graph on which reliability is a single traversal [17, 18]. The approach handles systems with thousands of variables. However, its cost is concentrated in the size of the diagram, which depends on the order in which the variables are decided, and finding a good ordering is itself intractable in general [19]. Reconvergent fan-out is the hard case for a diagram, since intermediate nodes must remember which shared ancestors have already been decided, and on a densely reconvergent network the construction can exhaust memory under one ordering while staying compact under another. Diagram implementations store each distinct sub-diagram once, a technique known as hash-consing [20, 21], and the same technique is used in Chapter 4 for a different unit of structure.

The probabilistic-inference literature reaches the same computation from another direction. Reachability from a source is marginal inference on a directed graphical model, and the exact inference methods are conditioning and clustering. Cutset conditioning enumerates the states of a set of nodes whose fixing renders the remaining structure singly connected [22], and the junction-tree algorithm clusters the network into a tree of overlapping bags [23]. Both are exact and both cost exponential in a width of the graph, the cutset size for the one and the treewidth for the other. Message passing makes the same inference local and scalable when exactness is relaxed. The directed probability propagation method of Tong and Tien is a reliability-specific variant that tracks marginal and pairwise probabilities to approximate the dependence between paths [24], gaining scale at the cost of approximation error and of pairwise bookkeeping that grows with the number of dependent interactions. Its sixteen-node benchmark grid is used again in Chapter 5.

Every method in this section, from the block diagram to the junction tree, takes a single number for each component probability. Section 0.5 will further discuss this.

0.3 Network Flow

Flow along capacitated edges is the oldest of the three analyses in its deterministic form. The maximum-flow minimum-cut theorem of Ford and Fulkerson states that the largest amount that can be sent from a source to a sink equals the smallest total capacity of any set of edges whose removal separates them [25, 26]. Hence the theorem gives a certificate of optimality, the minimum cut, alongside the flow itself, and the augmenting-path, blocking-flow and push-relabel families of algorithms that followed all compute both. Two further results from the same theory are used in Chapter 6. A node with its own capacity is handled by splitting it into an entry node and an exit node joined by an edge of that capacity, which preserves the correspondence between flows in the two networks, and any feasible flow decomposes into a set of source-to-sink path flows that sum to the total. The minimum cuts of a network are not in general unique, and their structure is characterised by the sets of nodes reachable from the source in the residual graph and reachable to the sink in the reverse residual graph, between which every minimum cut lies. The Birnbaum importance of a component, its marginal effect on system performance, was defined in the reliability setting [27] and carries over to flow as the marginal value of a unit of capacity.

Deterministic flow analysis does not address what happens when the capacities themselves are uncertain. Two different approaches have been developed for this case. In robust optimisation an uncertain capacity is treated adversarially: the capacity is taken from a set of possible values, no probability is assigned within that set, and the network is evaluated against the worst case in the set. The robust maximum-flow problem is solvable in polynomial time, while the corresponding robust minimum-cut problem is NP-hard [28]. The stochastic-flow network reliability problem, introduced by Doulliez and Jamoulle, holds each arc capacity as a discrete random variable and asks for the probability that the network can deliver a stated demand, computed by decomposing the space of feasible capacity configurations [29]. The family of methods that followed, state-space decomposition, boundary-point enumeration and sum-of-disjoint-products techniques, mirrors the path and cut-set methods of Section 0.2 closely, and the same formalism carries into project scheduling, where an activity's duration and cost take the place of an arc's capacity states (Section 0.6).

0.4 Scheduling on Networks

The critical path method models a project as a network of activities and computes the longest path through it. The project cannot finish earlier than that path allows, and the activities on it have no room to slip. Every other activity carries a float, equal to how far it can slip before it delays the project. The method and its float calculus date to the

late 1950s [30]. The question of duration uncertainty was raised almost at once. PERT replaced each fixed duration with a three-point estimate and a distributional assumption [31], and its descendants in modern practice replace the assumption with Monte Carlo sampling.

A distribution-free treatment of the same uncertainty came much later. When each duration is known only as an interval, the forward quantities remain easy, since the earliest completion of every node is a monotone function of the durations and its bounds are found at the two corners of the input box. However, the backward quantities change character. Deciding whether an activity is possibly critical is NP-complete, and computing float bounds is NP-hard [32], even on planar networks [33]. Exact methods exist for latest starting times and for one of the two float bounds [34], the consolidated picture of what is and is not tractable is given by Fortin, Zieliński, Dubois and Fargier [35], and the recognised tractable class is the series-parallel networks [36]. The backward quantities are hard because a float is the difference between two quantities, the project completion and the completion forced through the activity, which are both monotone in the durations but share the same inputs. Their difference is therefore not monotone, and the corner argument that works for the forward quantities does not apply.

Furthermore, none of the machinery is specific to time. The forward recursion combines incoming values and compounds them along edges, and any quantity with those two operations propagates the same way. With combination as maximum and compounding as addition the recursion is the critical path method. With other pairs it is cost roll-up, risk scaling or route selection. Max-plus linear systems theory treats the classical forward and backward passes as linear algebra over an idempotent semiring, with the backward pass arising as residuation [37]. A forward recursion exists for any pair of operations, but a backward one is a property the pair either has or lacks, and this is the discipline Chapter 7 adopts.

0.5 Representing Uncertain Inputs

The methods reviewed so far take a component failure probability, an edge capacity and an activity duration as known numbers. In practice each is estimated from sparse failure data, elicited from expert judgement, or quoted as a manufacturer's tolerance. Chapter 1 distinguished aleatory uncertainty, the variability of a quantity across a population or over repeated trials, from epistemic uncertainty, a lack of knowledge about a quantity that more data would reduce. Probability theory is the accepted framework for the first. For the second it is less satisfactory, because it requires a precise probability to be assigned even when the evidence supports only a range, so that a quantity about which little is known is modelled in the same way as one that is known well. Generalised probabilistic methods have been developed to describe epistemic uncertainty with fewer assumptions,

and they can be applied to any computational model without modifying it.

0.5.1 Intervals and probability boxes

An interval $[x, \bar{x}]$ states that a quantity lies between two bounds and states nothing else. It is the appropriate form when a datasheet gives a tolerance, when an expert will commit to a range but not to a number, or when a value is known only to a measurement precision. Interval arithmetic returns an enclosure of the true range of a function over the interval inputs. The enclosure is exact only when each input variable appears once in the expression. Otherwise the arithmetic treats repeated occurrences as if they were independent, and the enclosure is wider than the true range. This is known as the dependency problem. For a function that is monotone in each of its inputs the exact range is available without interval arithmetic at all, from two evaluations at the lower and upper corners of the input box. This observation is used in Chapters 5 and 7.

A probability box, or p-box, bounds a distribution instead of a value. Following Ferson et al. [38], a p-box on a random variable X is a pair of cumulative distribution functions $\underline{F}$ and $\overline{F}$ with $\underline{F}(x) \leq \overline{F}(x)$ for all x , together with optional constraints on the moments and on the family of the distribution. A distribution F belongs to the p-box when $\underline{F}(x) \leq F(x) \leq \overline{F}(x)$ for all x , its mean and variance lie in the stated intervals, and it belongs to the stated family if one is given. When the two bounding functions coincide the p-box is a single distribution and represents aleatory uncertainty alone; when they are the step functions at the two ends of an interval the p-box is that interval and represents epistemic uncertainty alone; between these two cases the p-box carries both. A p-box therefore covers the situation of a distribution whose family is known but whose parameters are not, and the situation of a distribution estimated from samples measured with limited precision.

Williamson and Downs established the arithmetic of p-boxes [39]. For two random variables with p-boxes A and B and a binary operation, the p-box of the result depends on what is known about their dependence. Under independence the result is a convolution of the bounding functions, evaluated on a discretisation of each into a fixed number of levels, so that the cost of every operation grows with the discretisation. Under no assumption about dependence the result is obtained from the Fréchet–Hoeffding inequalities, which bound the joint distribution of any pair of random variables by functions of their marginals alone. The Fréchet bound is valid whatever the true dependence and is therefore sound. For the same reason it is wider than the independence bound. Which of the two applies at a given step depends on whether the quantities being combined are functions of disjoint sets of underlying variables, and this question governs the p-box propagation of Chapter 5. Software for p-box arithmetic exists in R [40], Python [41] and Julia [42]; the Julia library is one of the libraries this framework builds on.

0.5.2 Neighbouring theories

Two other theories represent epistemic uncertainty without a precise probability assignment. Fuzzy set theory represents a quantity by a membership function, assigning each possible value a graded degree of membership instead of a frequency [43], and possibility theory develops the same idea into a pair of measures, possibility and necessity. Evidence theory, also called Dempster–Shafer theory, assigns belief mass to sets of values directly and combines evidence from independent sources without assigning a probability to individual outcomes [44]; its belief and plausibility functions bound the probability of an event from below and above. Both theories can be placed within the wider field of imprecise probability, and a Dempster–Shafer structure on the real line can be converted to a p-box and back. Neither theory is used in this thesis. The quantities propagated here are probabilities, capacities and durations, and the information available on them in practice, such as a range from a datasheet or a distribution estimated from sparse samples, is stated directly as an interval or a p-box without translation into membership functions or belief masses. In addition, the operations the framework performs on its values are products, complements and weighted combinations, for which the p-box arithmetic above gives results that are again p-boxes, so the same representation is carried from input to output without conversion.

0.5.3 Imprecision in the three analyses

The exact reliability methods of Section 0.2 all consume a point-valued component probability, and none of them propagates an interval or a p-box natively. For example, an interval query against a decision diagram means evaluating the diagram at chosen input corners, and there is no analytic route from a diagram to a bound on a distribution. In flow analysis, Section 0.3 showed the two established responses to uncertain capacities, the adversarial uncertainty set of robust optimisation and the point-valued discrete random variable of stochastic-flow reliability, and neither is an interval or a p-box in the sense above. In scheduling, the interval treatment of durations exists and its complexity is known, and the difficulty is confined to the backward quantities. Across the three analyses, the same small set of representations recurs, point values, distributions, adversarial sets and intervals, but no single representation is used consistently across all three. Each analysis developed its own treatment of uncertainty independently of the other two.

0.6 Multi-Analysis Tools for Complex Infrastructure

As discussed in Chapter 1, the same network is often analysed for several different behaviours, and several engineering software tools run more than one kind of analysis against a single network representation. `pandapower`, an open-source power-system tool, runs

power flow, optimal power flow, state estimation, topological graph search and short-circuit calculation against one explicit graph of buses, lines and transformers, a range of analyses matched by commercial suites such as DIgSILENT PowerFactory and PSS/E [45]. The ArcGIS Utility Network models power, gas and water networks as a graph of features connected through topology and runs eight distinct trace types, connectivity, isolation, subnetwork discovery and loop detection among them, against that shared topology [46]. Neither tool’s analyses are source-to-node reliability, capacitated flow or activity scheduling, and neither propagates interval or p-box uncertainty.

Whether an existing tool or method combines these three analyses specifically, on one network representation, is a narrower question. A search for such a tool found three partial answers among tools and methods and one comparable piece of doctoral research, and none of the four meets the description fully.

Commercial reliability software has, in practice, paired reliability with flow. Several of the block diagram and fault tree suites in active use, ReliaSoft’s BlockSim, Isograph’s Availability Workbench and ITEM Software’s ITEM ToolKit among them, bundle a throughput or process-flow module alongside their reliability engine, modelling multi-flow-type capacity, bottlenecks and resource allocation on the same diagrams the reliability analyses use [47–49]. None of their public documentation describes a project-scheduling or critical-path module, and all three represent component life and failure with classical distributions, fitted from data or specified by the analyst.

Open-source infrastructure simulation platforms have paired flow with a form of scheduling. InfraRisk integrates power-flow, water-hydraulic and traffic-flow simulators for interdependent power, water and road networks under a resilience framing [50]. Its recovery module sequences repair-crew actions with start and end times computed from travel time and repair duration, by heuristic prioritisation or by model-predictive-control optimisation. This sequencing is a logistics schedule, since it optimises the order and timing of repair actions against a performance-loss metric. It is not a critical-path computation over a fixed activity network. Similarly, its reliability side is not a source-to-node reachability computation. The hazard module derives point component-failure probabilities from exposure and intensity to drive Monte Carlo disaster scenarios, and the platform reports its results as simulated performance loss.

A third direction pairs reliability with schedule directly, inside project-network reliability itself. Modelling a project’s activities as the arcs of a multi-state network and asking for the probability that the project completes within both a time and a budget constraint is an established line of work. Huang, Huang and Lin compute this project reliability exactly. The space of feasible completion-time and budget combinations is split into subsets via the project’s upper and lower boundary points, each subset is evaluated by inclusion-exclusion, and the method is validated against full enumeration on a thirteen-activity manufacturing project [51]. However, the method has no notion of a

physical are capacity carrying a transportable flow, since activity duration and cost stand in its place, and every input is a discrete state with a fixed probability.

Tong’s doctoral thesis comes closest to combining two of the three exactly. The author develops new methods for infrastructure flow networks and investigates their reliability in terms of both connectivity and flow capacity, using the probability propagation method reviewed in Section 0.2 [52]. Schedule and critical-path analysis are absent from it, and the probability treatment throughout is point-valued.

In summary, no candidate combines all three analyses on one network. Reliability and flow are paired in commercial software with no schedule dimension. Flow and a form of scheduling are paired in open-source simulation platforms with no formal reliability computation. Reliability and schedule are paired in project-network theory with no flow network. Reliability and flow are paired again, this time exactly, in Tong’s thesis, without schedule. Every one of the four represents its uncertain inputs precisely or adversarially.

0.7 Delivery of Engineering Analysis Software

The gap between an analysis existing in a library and an engineer being able to use it is sometimes referred to as the last-mile problem [53, 54]. Engineering analysis software is generally delivered to its users in one of three ways, as a browser-based service, as a licensed desktop installation, or as a web-based geographic information system. None of these is what the business-software industry calls no-code, and this section reviews all four.

The no-code and low-code software category defines itself around business application development. A systematic review of 383 definitions across 306 publications from 2014 to 2025 characterises no-code platforms as visual, drag-and-drop environments that let a non-programmer build software without writing code, and low-code platforms as the same idea with a professional-developer audience layered on top [55]. The same review maps the identified platforms onto ten functional classes, including enterprise resource planning extensions, customer relationship platforms, business process automation suites, robotic process automation, document management and integration hubs, with commercial examples such as Mendix, Salesforce, UiPath, Bizagi and Zapier. Engineering or scientific analysis does not appear among them.

Within engineering analysis, browser-delivered simulation as a service is one established pattern. SimScale, launched in 2013, runs computational fluid dynamics, finite element, thermal and electromagnetic analysis in the browser with no local installation [56], with the computation running in the vendor’s cloud. The licensed desktop suite is a second and older pattern. The commercial reliability software named in Section 0.6 requires no programming and runs as a local installation, so it has the local computation that SimScale lacks and lacks the browser delivery that SimScale has. Web-based

geographic information systems are a third. A PRISMA-based systematic review of web GIS for hazard and infrastructure risk assessment screened 1,775 articles and admitted 65. It organises the field around themes including visualisation, user-interface design and decision support [57]. The platform of Repetto et al. is a representative member: it delivers wind monitoring, forecasting and statistical risk mapping for ports, motorways, rail lines and industrial plants, through a web GIS built on open geospatial software, operated by port authorities who never touch the underlying numerical models [58]. Most platforms in this category address monitoring and forecasting. They do not perform a structural reliability or component-failure computation. Similarly, network visualisation environments such as Gephi [59] and Cytoscape [60] make graphs visual and explorable but carry no quantitative reliability analysis. The analytical capability lives in libraries, e.g. in Matlab [61], R [40], Julia [42] and Python [41], which a non-programmer cannot use.

A separate obstacle is data sovereignty. The accessible form of modern software is the web application, and the usual web application is a cloud service, which raises security and custody questions for proprietary system models [62], and in domains such as nuclear decommissioning is not acceptable at all. Local-first software keeps the usability of the web while storing and computing its data on the user’s own machine [63]. SimScale is browser-based but not local; the desktop suites are local but not browser-based; the web GIS platforms are browser-based but run on a remote server. No tool found in this search combines browser delivery, no programming, and local computation and data.

0.8 Graph Tooling Across Languages

General-purpose graph libraries exist in every language the work could have been written in. MATLAB has offered shortest-path, maximum-flow, centrality, connected-component and spanning-tree functions on its native graph classes since the mid-2010s [64], and R’s `igraph` provides the same over one shared graph object [65]. Python’s `NetworkX` covers simple, directed and multi-edge graphs over a single native numeric type [66], and Rust’s `petgraph` is a newer and smaller equivalent [67]. Julia’s ecosystem, centred on `Graphs.jl` and its satellite packages for weighted and property graphs, occupies the same position [68]. Dedicated reliability tooling is rarer. R carries a package built for structural reliability, computing system and survival signatures over a network’s topology [69]. No single Python package built for network reliability in the block diagram, path-set or decision-diagram sense was found; the closest candidate, `pgmpy`, is a general Bayesian-network toolkit. This is a gap in packaged tooling only, since network reliability is an actively reviewed field with applications across communications, transportation and energy distribution [70]. Furthermore, none of the packages named above, in any of the five languages, carries a native notion of an interval- or p-box-valued edge or node,

because none is built for propagating uncertain quantities across a network. The reasons for choosing Julia, and the language features the implementation depends on, are given in Chapter 8.

0.9 Summary

Network reliability, network flow and scheduling analysis developed separately, each with its own exact methods and its own treatment of uncertain inputs. The exact reliability methods are all exponential in some width of the network and all consume a point-valued component probability. Interval and p-box representations of epistemic uncertainty exist, with an established arithmetic, but are not propagated natively by any of the exact methods reviewed. Several engineering software tools run more than one analysis on one network object, but no tool, method or thesis found in this search combines source-to-node reliability, capacitated flow and activity scheduling on one network representation, and none of the closest candidates propagates imprecision. No engineering analysis tool found combines browser delivery, no programming and local computation. The framework developed in the following chapters addresses each of these points.

Bibliography

- [1] L. G. Valiant, “The complexity of enumeration and reliability problems,” *SIAM Journal on Computing*, vol. 8, no. 3, pp. 410–421, 1979. DOI: 10.1137/0208032
- [2] M. O. Ball, “Computational complexity of network reliability analysis: An overview,” *IEEE Transactions on Reliability*, vol. R-35, no. 3, pp. 230–239, 1986. DOI: 10.1109/TR.1986.4335422
- [3] R. Billinton and R. N. Allan, “Network modelling and evaluation of simple systems,” in *Reliability Evaluation of Engineering Systems: Concepts and Techniques*, Boston, MA: Springer US, 1992.
- [4] L. Xing and S. V. Amari, “Fault tree analysis,” in *Handbook of Performability Engineering*, London: Springer London, 2008.
- [5] M. Ouyang, “Review on modeling and simulation of interdependent critical infrastructure systems,” *Reliability Engineering & System Safety*, vol. 121, pp. 43–60, 2014. DOI: 10.1016/j.ress.2013.06.040
- [6] S. V. Buldyrev, R. Parshani, G. Paul, H. E. Stanley and S. Havlin, “Catastrophic cascade of failures in interdependent networks,” *Nature*, vol. 464, no. 7291, pp. 1025–1028, 2010. DOI: 10.1038/nature08932
- [7] W. Liu and J. Li, “An improved cut-based recursive decomposition algorithm for reliability analysis of networks,” *Earthquake Engineering and Engineering Vibration*, vol. 11, no. 1, pp. 1–10, 2012.
- [8] Y. Kim and W.-H. Kang, “Network reliability analysis of complex systems using a non-simulation-based method,” *Reliability Engineering & System Safety*, vol. 110, pp. 80–88, 2013.
- [9] A. Satyanarayana and R. K. Wood, “A linear-time algorithm for computing K-terminal reliability in series-parallel networks,” *SIAM Journal on Computing*, vol. 14, no. 4, pp. 818–832, 1985. DOI: 10.1137/0214057
- [10] E. F. Moore and C. E. Shannon, “Reliable circuits using less reliable relays,” *Journal of the Franklin Institute*, vol. 262, no. 3, pp. 191–208, 1956. DOI: 10.1016/0016-0032(56)90559-2

- [11] A. Satyanarayana and M. K. Chang, “Network reliability and the factoring theorem,” *Networks*, vol. 13, no. 1, pp. 107–120, 1983. DOI: 10.1002/net.3230130107
- [12] L. B. Page and J. E. Perry, “A practical implementation of the factoring theorem for network reliability,” *IEEE Transactions on Reliability*, vol. 37, no. 3, pp. 259–267, 1988. DOI: 10.1109/24.3752
- [13] R. K. Wood, “A factoring algorithm using polygon-to-chain reductions for computing K-terminal network reliability,” *Networks*, vol. 15, no. 2, pp. 173–190, 1985. DOI: 10.1002/net.3230150204
- [14] N. Robertson and P. D. Seymour, “Graph minors. II. Algorithmic aspects of treewidth,” *Journal of Algorithms*, vol. 7, no. 3, pp. 309–322, 1986. DOI: 10.1016/0196-6774(86)90023-4
- [15] A. K. Goharshady and F. Mohammadi, “An efficient algorithm for computing network reliability in small treewidth,” *Reliability Engineering & System Safety*, vol. 193, p. 106665, 2020. DOI: 10.1016/j.ress.2019.106665
- [16] G. S. Fishman, “A comparison of four Monte Carlo methods for estimating the probability of s–t connectedness,” *IEEE Transactions on Reliability*, vol. 35, no. 2, pp. 145–155, 1986.
- [17] R. E. Bryant, “Graph-based algorithms for Boolean function manipulation,” *IEEE Transactions on Computers*, vol. C-35, no. 8, pp. 677–691, 1986. DOI: 10.1109/TC.1986.1676819
- [18] A. Rauzy, “New algorithms for fault trees analysis,” *Reliability Engineering & System Safety*, vol. 40, no. 3, pp. 203–211, 1993.
- [19] R. E. Bryant and C. Meinel, “Ordered binary decision diagrams,” in *Logic Synthesis and Verification*, Boston, MA: Springer, 2002.
- [20] A. P. Ershov, “On programming of arithmetic operations,” *Communications of the ACM*, vol. 1, no. 8, pp. 3–6, 1958, The origin of the hash-consing technique. DOI: 10.1145/368892.368907
- [21] J.-C. Filliâtre and S. Conchon, “Type-safe modular hash-consing,” in *Proceedings of the ACM SIGPLAN Workshop on ML*, 2006, pp. 12–19. DOI: 10.1145/1159876.1159880
- [22] J. Pearl, *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. San Francisco, CA: Morgan Kaufmann, 1988.
- [23] S. L. Lauritzen and D. J. Spiegelhalter, “Local computations with probabilities on graphical structures and their application to expert systems,” *Journal of the Royal Statistical Society, Series B*, vol. 50, no. 2, pp. 157–224, 1988.

- [24] Y. Tong and I. Tien, “Probability propagation method for reliability assessment of acyclic directed networks,” *ASCE-ASME Journal of Risk and Uncertainty in Engineering Systems, Part A: Civil Engineering*, vol. 5, no. 3, p. 04019011, 2019.
- [25] L. R. Ford and D. R. Fulkerson, “Maximal flow through a network,” *Canadian Journal of Mathematics*, vol. 8, pp. 399–404, 1956. DOI: 10.4153/CJM-1956-045-5
- [26] L. R. Ford and D. R. Fulkerson, *Flows in Networks*. Princeton, NJ: Princeton University Press, 1962.
- [27] Z. W. Birnbaum, “On the importance of different components in a multicomponent system,” in *Multivariate Analysis II*, P. R. Krishnaiah, Ed., New York: Academic Press, 1969, pp. 581–592.
- [28] D. Bertsimas, E. Nasrabadi and S. Stiller, “Robust and adaptive network flows,” *Operations Research*, vol. 61, no. 5, pp. 1218–1242, 2013. DOI: 10.1287/opre.2013.1200
- [29] P. Doulliez and E. Jamoulle, “Transportation networks with random arc capacities,” *Revue française d’automatique, informatique, recherche opérationnelle. Recherche opérationnelle*, vol. 6, no. V3, pp. 45–59, 1972. [Online]. Available: https://www.numdam.org/item/R0_1972__6_3_45_0/
- [30] J. E. Kelley and M. R. Walker, “Critical-path planning and scheduling,” in *Proceedings of the Eastern Joint IRE-AIEE-ACM Computer Conference*, Boston, MA, 1959, pp. 160–173. DOI: 10.1145/1460299.1460318
- [31] D. G. Malcolm, J. H. Roseboom, C. E. Clark and W. Fazar, “Application of a technique for research and development program evaluation,” *Operations Research*, vol. 7, no. 5, pp. 646–669, 1959. DOI: 10.1287/opre.7.5.646
- [32] S. Chanas and P. Zieliński, “The computational complexity of the criticality problems in a network with interval activity times,” *European Journal of Operational Research*, vol. 136, no. 3, pp. 541–550, 2002. DOI: 10.1016/S0377-2217(01)00048-0
- [33] S. Chanas and P. Zieliński, “On the hardness of evaluating criticality of activities in a planar network with duration intervals,” *Operations Research Letters*, vol. 31, no. 1, pp. 53–59, 2003.
- [34] D. Dubois, H. Fargier and J. Fortin, “Computational methods for determining the latest starting times and floats of tasks in interval-valued activity networks,” *Journal of Intelligent Manufacturing*, vol. 16, pp. 407–421, 2005. DOI: 10.1007/s10845-005-1654-5

- [35] J. Fortin, P. Zieliński, D. Dubois and H. Fargier, “Criticality analysis of activity networks under interval uncertainty,” *Journal of Scheduling*, vol. 13, pp. 609–627, 2010. DOI: 10.1007/s10951-010-0163-3
- [36] J. Valdes, R. E. Tarjan and E. L. Lawler, “The recognition of series parallel digraphs,” *SIAM Journal on Computing*, vol. 11, no. 2, pp. 298–313, 1982. DOI: 10.1137/0211023
- [37] F. Baccelli, G. Cohen, G. J. Olsder and J.-P. Quadrat, *Synchronization and Linearity: An Algebra for Discrete Event Systems*. Chichester: John Wiley & Sons, 1992.
- [38] S. Ferson, V. Kreinovich, L. Ginzburg, D. S. Myers and K. Sentz, “Constructing probability boxes and Dempster–Shafer structures,” Sandia National Laboratories, Albuquerque, NM, Tech. Rep. SAND2002-4015, 2003.
- [39] R. C. Williamson and T. Downs, “Probabilistic arithmetic I: Numerical methods for calculating convolutions and dependency bounds,” *International Journal of Approximate Reasoning*, vol. 4, no. 2, pp. 89–158, 1990.
- [40] S. Ferson, *Pba.r: Probability bounds analysis in R*, <https://github.com/ScottFerson/pba.r>, 2019.
- [41] N. Gray, S. Ferson, M. De Angelis, A. Gray and F. Baumont de Oliveira, “Probability bounds analysis for Python,” *Software Impacts*, vol. 12, p. 100 246, 2022. DOI: 10.1016/j.simpa.2022.100246
- [42] A. Gray, S. Ferson and E. Patelli, *ProbabilityBoundsAnalysis.jl: Arithmetic with sets of distributions*, Proceedings of JuliaCon, 2021. [Online]. Available: <https://github.com/Institute-for-Risk-and-Uncertainty/ProbabilityBoundsAnalysis.jl>
- [43] L. A. Zadeh, “Fuzzy sets,” *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.
- [44] G. Shafer, *A Mathematical Theory of Evidence*. Princeton, NJ: Princeton University Press, 1976.
- [45] L. Thurner et al., “Pandapower—an open-source python tool for convenient modeling, analysis, and optimization of electric power systems,” *IEEE Transactions on Power Systems*, vol. 33, no. 6, pp. 6510–6521, 2018. DOI: 10.1109/TPWRS.2018.2829021
- [46] Esri, *ArcGIS utility network: Trace types*, <https://doc.esri.com/en/arcgis-pro/latest/help/data/utility-network/utility-network-trace-types.html>, 2024.
- [47] Hottinger Brüel & Kjær (ReliaSoft). “BlockSim: System reliability, availability, and maintainability analysis.”

- [48] Isograph Ltd. “Availability Workbench.”
- [49] ITEM Software. “ITEM ToolKit: Reliability analysis and safety software tools.”
- [50] S. Balakrishnan and B. Cassottana, “InfraRisk: An open-source simulation platform for resilience analysis in interconnected power–water–transport networks,” *Sustainable Cities and Society*, vol. 83, p. 103 963, 2022. DOI: 10.1016/j.scs.2022.103963
- [51] D.-H. Huang, C.-F. Huang and Y.-K. Lin, “Exact project reliability for a multi-state project network subject to time and budget constraints,” *Reliability Engineering & System Safety*, vol. 195, p. 106 744, 2020. DOI: 10.1016/j.ress.2019.106744
- [52] Y. Tong, “New approaches for modeling and reliability assessment of infrastructure flow networks,” Ph.D. dissertation, Georgia Institute of Technology, 2021.
- [53] J. E. Hannay, C. MacLeod, J. Singer, H. P. Langtangen, D. Pfahl and G. Wilson, “How do scientists develop and use scientific software?” In *Proceedings of the 2009 ICSE Workshop on Software Engineering for Computational Science and Engineering*, IEEE, 2009, pp. 1–8. DOI: 10.1109/SECSE.2009.5069155
- [54] P. Prabhu et al., “A survey of the practice of computational science,” in *Proceedings of the 2011 International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC ’11, New York, NY: ACM, 2011, pp. 1–12. DOI: 10.1145/2063348.2063374
- [55] A. Abendroth, “The democratization of software engineering: Evolution, definition, and the future of low-code and no-code platforms,” *Management Review Quarterly*, vol. 67, 2026. DOI: 10.1007/s11301-026-00603-2
- [56] SimScale GmbH, *SimScale: AI-native engineering simulation software in the cloud*, <https://www.simscale.com/>, 2026.
- [57] M. Daud, F. M. Ugliotti and A. Osello, “Comprehensive analysis of the use of web-gis for natural hazard management: A systematic review,” *Sustainability*, vol. 16, no. 10, p. 4238, 2024. DOI: 10.3390/su16104238
- [58] M. P. Repetto, M. Burlando, G. Solari, P. De Gaetano, M. Pizzo and M. Tizzi, “A web-based GIS platform for the safe management and risk assessment of complex structural and infrastructural systems exposed to wind,” *Advances in Engineering Software*, vol. 117, pp. 29–45, 2018. DOI: 10.1016/j.advengsoft.2017.03.002
- [59] M. Bastian, S. Heymann and M. Jacomy, “Gephi: An open source software for exploring and manipulating networks,” in *Proceedings of the International AAAI Conference on Web and Social Media*, vol. 3, 2009, pp. 361–362. DOI: 10.1609/icwsm.v3i1.13937

- [60] P. Shannon et al., “Cytoscape: A software environment for integrated models of biomolecular interaction networks,” *Genome Research*, vol. 13, no. 11, pp. 2498–2504, 2003. DOI: 10.1101/gr.1239303
- [61] E. Patelli, H. George-Williams, J. Sadeghi, R. Rocchetta, M. Broggi and M. de Angelis, “OpenCossan 2.0: An efficient computational toolbox for risk, reliability and resilience analysis,” in *Proceedings of the Joint ICVRAM ISUMA UNCERTAINTIES Conference*, A. T. Beck, G. F. M. de Souza and M. A. Trindade, Eds., 2018.
- [62] K. Hashizume, D. G. Rosado, E. Fernández-Medina and E. B. Fernández, “An analysis of security issues for cloud computing,” *Journal of Internet Services and Applications*, vol. 4, no. 1, pp. 1–13, 2013. DOI: 10.1186/1869-0238-4-5
- [63] M. Kleppmann, A. Wiggins, P. van Hardenberg and M. McGranaghan, “Local-first software: You own your data, in spite of the cloud,” in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, ser. Onward! 2019, ACM, 2019, pp. 154–178. DOI: 10.1145/3359591.3359737
- [64] The MathWorks, Inc., *Graph and network algorithms*, <https://www.mathworks.com/help/matlab/graph-and-network-algorithms.html>, 2026.
- [65] G. Csárdi and T. Nepusz, “The igraph software package for complex network research,” *InterJournal, Complex Systems*, p. 1695, 2006.
- [66] A. A. Hagberg, D. A. Schult and P. J. Swart, “Exploring network structure, dynamics, and function using NetworkX,” in *Proceedings of the 7th Python in Science Conference (SciPy2008)*, G. Varoquaux, T. Vaught and J. Millman, Eds., Pasadena, CA, 2008, pp. 11–15. DOI: 10.25080/TCWV9851
- [67] petgraph contributors, *Petgraph: Graph data structure library for Rust*, <https://github.com/petgraph/petgraph>, 2026.
- [68] J. Bezanson, A. Edelman, S. Karpinski and V. B. Shah, “Julia: A fresh approach to numerical computing,” *SIAM Review*, vol. 59, no. 1, pp. 65–98, 2017. DOI: 10.1137/141000671
- [69] L. J. M. Aslett, *ReliabilityTheory: Tools for structural reliability analysis*, <https://cran.r-project.org/package=ReliabilityTheory>, 2024.
- [70] V. Gaur, O. P. Yadav, G. Soni and A. P. S. Rathore, “A literature review on network reliability analysis and its engineering applications,” *Proceedings of the Institution of Mechanical Engineers, Part O: Journal of Risk and Reliability*, vol. 235, no. 2, pp. 167–181, 2021.